Software Defined Networks Key Relay for Large-Scale Quantum Key Distribution Networks

Stephan Laschet* ORCID: 0009-0003-1142-8870
Gergely Lendvay* ORCID: 0009-0003-9228-189X
Thomas Lorünser* ORCID: 0000-0002-1829-4882
Paul James* ORCID: 0009-0008-7046-2151
Luca Torresetti* ORCID: 0009-0002-1257-061X
Alessandro Colombo* ORCID: 0009-0009-4574-7101
*Center for Digital Safety and Security, AIT Austrian Institute of Technology, Vienna, Austria

arXiv:2608.17539v1 [cs.CR] 18 Aug 2026

***Abstract*—This work addresses the orchestration of large-scale Quantum Key Distribution Networks (QKDNs) using Software Defined Networking (SDN). Building on ETSI and ITU specifications, common best practices and architectures are outlined. The main task of the SDN Controller is to aggregate technical key performance indicators (KPI) from the network and, based on these, select the optimal path. Multiple path selection algorithms, based on Dijkstra or a maximum-minimum capacity algorithm, with built-in load balancing are presented. The algorithms were tested in simulations and their performances, and tradeoffs, are discussed. Additional critical aspects related to SDN controlled QKDNs are discussed, such as query batching, multi-path selection and group key capabilities. An oblivious multi-party protocol is proposed for relay path selection in a multi-domain scenario, so providers don't have to disclose sensitive information about their QKDN. These contributions aim to enhance scalability, resilience and interoperability in quantum-secure network infrastructures.**

I. Introduction

As quantum computer maturity progresses [1], it threatens commonly used asymmetric cryptography, since it enables efficient attacks of mathematical cryptographic foundations, using Shor's algorithm [2]. To counteract this threat, one solution is Quantum Key Distribution (QKD) [3] which does not only withstand quantum-driven attacks, but also any future technological advancement. Instead of relying on computational hardness assumptions, QKD establishes a symmetric cryptographic key based on the laws of quantum physics, more specifically, on the non-cloning theorem [4], [5] and Information Theoretically Secure (ITS) algorithms [6].

But QKD comes with some unique characteristics that need to be accounted for. The limitation that only directly connected parties within a certain range can establish a key is in stark contrast to the technology QKD is supposed to replace.

To overcome this, QKD links are combined to a QKD Network (QKDN). The QKD pairs still only establish point-to-point keys, but a Key Management System (KMS) layer above uses them and ITS-algorithms to establish end-to-end keys between any party in the QKDN. Different techniques exist to relay the keys [7], all requiring a continuous path through the network. The Software Defined Networking (SDN) [8], [9] approach seems to be one of the most promising candidates for managing QKDNs, many ETSI specifications utilize this technology, as elaborated later. Not only does SDN enable a holistic network view for relay path selection, it allows additional use-cases.

Especially for multi-domain QKDNs such as EuroQCI [10], a consistent path calculation in different domains is of upmost importance. However, as shown in Section III, those topics are underrepresented in specifications and publications, therefore the authors' findings and approaches are presented in this work.

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works. Published version: https://doi.org/10.1109/QCNC69040.2026.00115

II. Contribution

The state of the art related to architecture, KPIs and path finding is discussed in Section III. Given the identified gaps, this work contributes by presenting the authors' findings related to metrics in Section IV and path finding algorithms in Section V, including results from a simulation framework [11]. This work also addresses the gap in SDN Agent to KMS interface specification by publishing an OpenAPI specification [12]. Further open issues, such as multi-path and group key support, are discussed in Section VI. Finally, in the light of multi-domain QKDNs, oblivious inter-domain routing is proposed in Section VII.

III. State of the Art and Related Work

A. Related work at ETSI GS QKD

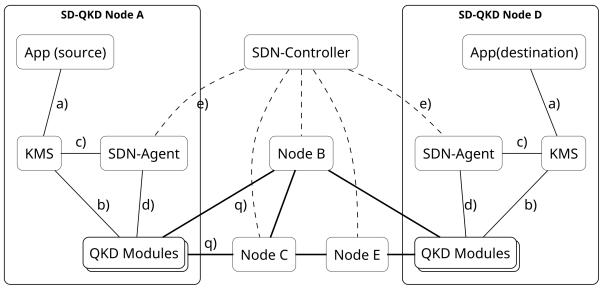


Figure 1: ETSI GS QKD 015 [13] SDN managed QKDN. Node A and D are fully shown, others compressed.

The ETSI GS QKD 015 specification [13] focuses on the use of SDN in QKDNs and proposes a high-level architecture as depicted in Fig. 1. It mainly defines an interface between the SDN Controller and the SDN Agent (Fig. 1, e). The SDN Agent represents and models the SD-QKD Node, which is composed of the KMS and QKD modules. It forwards the SD-QKD Node data to the SDN Controller and receives configuration data from it to distribute within the Node.

Other SDN related ETSI GS QKD specifications are:

- ETSI GS QKD 018 [14] SDN for coexistence of classical and quantum channels.
- ETSI GS QKD 021 Draft [15] SDN interface to manage separated QKDNs.
- ETSI GS QKD 023 Draft [15] SDN interface to monitor the KPIs of QKD modules.

The SDN Agent to KMS API (Fig. 1, c) lacks specification as mentioned in previous publications [16]. To address the lack of specification, the authors' OpenAPI specification [12] is released in the context of this paper.

B. Related work at ITU-T

The main ITU-T recommendations are Y.3804 [17] and Y.3805 [18], which describe QKDN monitoring and control.

C. QKDN application interfaces

The application interfaces (Fig. 1, a) must be considered for designing the SDN as their behavior significantly determine the behavior of the QKDN.

1) ETSI GS QKD 014: This API [19] is commonly used for obtaining keys from the QKDN by an application. The application states the source and destination as well as the number and size of keys, which is the origin of that data for the SDN. The API is stateless and does not define a Quality of Service (QoS). Theoretically an application can ask for any peer as often and for as many keys as it likes.

It also defines a group key feature, for sharing keys with several peers.

2) ETSI GS QKD 004: In contrast, this API [20] is stateful using a session with clearly defined QoS, e.g. defining maximum key rates. Each session is only valid for a dedicated application pair.

3) API comparison: The ETSI GS QKD 014 is more accepted in the industry, since its stateless and unrestricted behavior works well for applications, but makes path selection and service upkeep more challenging. The ETSI GS QKD 004 is more restrictive to applications, but the session concept works better for SDN path selection and monitoring.

D. Related work on QKDN SDN relay

Some QKDNs are SDN managed, for example MadQCI [21], but details about their path finding algorithm are not disclosed, beyond it considering QBER and SKR [22]. The SeCoQC project [23] uses Dijkstra, but doesn't disclose the weighting functions. The DISCRETION project showed military SDN managed QKDNs, but does not disclose details on path finding algorithms [24]. The DemoQuanDT project published their path finding approach: They used Dijkstra with a weighting function considering the key store emptiness and the inverse of the SKR [25]. ADA-QKDN uses SPF with the weighting function equivalent to $1/\text{ESKR}$ [26].

IV. KPI SELECTION

Technical Key Performance Indicators (KPI) for path selection are discussed in this section. Mainly the KMS layer is considered, as it performs key relay and interacts with applications.

A. Related work on KPIs for SDN QKDN

The ETSI GS QKD 015 [13] defines three main KPIs: The *Secure Key Rate (SKR)* is the generation rate of the QKD link. The *Effective Secret Key Rate (ESKR)* is equal to the SKR minus internal key usage. The *Expected consumption* is the key rate queried by applications.

The ITU-T recommendations on QKDN QoS metrics, Y.3806 [27] and Y.3807 [28], focus on QKDN performance for applications, and are less suited for relay path finding.

B. KPI definitions

As the KPIs of Section IV-A were found to be suboptimal for path selection, this section introduces other KPIs. They are vendor independent and easy to determine for any QKDN.

Since each edge in the path must have enough keys stored to support the requested amount, obviously the SDN Controller must be aware of this metric. The ESKR defined in ETSI GS QKD 015 is impractical as it is a rate, not an absolute value. Only if the SDN was recording the ESKR for each edge from the beginning, the currently buffered keys could be calculated.

Therefore, the metric ***Key availability (KAV)*** is introduced. This metric gives the available key material in bytes on the specified QKD link, which could be delivered to applications. A KMS can trivially obtain this KPI from its database and publish it to the SDN. The ESKR can still be determined by calculating $\text{ESKR} = \Delta\text{KAV}/\Delta t$.

In a QKDN, frequently used edges should be avoided, due to the increased load making it more likely that this link runs out of keys. Hence, the KPI ***edge usage*** is introduced giving the number of relay paths using a specific edge. Third, the ***hop count*** is considered, measuring the number of nodes to be traversed in the specified path. Overall it is less relevant than in internet routing, however hops add delays, overall more keys are used for the relay process, and each hop exposes the relayed key to potential attacks on trusted nodes.

The most important metric is the KAV, as it is the scarcest resource, determines the feasibility of path establishment and the available reserve for future requests. The second is the edge usage for load balancing, the third is the path hop count. To summarize in priority order:

Key availability (KAV): This metric given in bytes determines the amount of key material at the KMS on the direct QKD link available for applications.

Edge usage: This metric given as unsigned integer determines, how many paths are using this direct QKD link.

Hop count: This metric given as unsigned integer determines how many nodes must be traversed to reach the destination.

Other metrics from the QKDN are still relevant for monitoring and (re-)configuration.

V. Path selection Algorithms

As discussed in Section III limited information is published on QKDN path finding, therefore this section outlines the authors' research on the matter. The authors acknowledge that the effectiveness of a particular algorithm, heuristic, or weighting strategy may vary for different network structures.

Graph theory is used for path selection, where the QKDN is represented by nodes incorporating the QKD, KMS and application instances. Nodes connected by an edge generate point-to-point QKD keys. An edge is assigned a weight, so it can be compared to other edges usable for a path. To generate the weights, the relevant KPIs of Section IV are compiled by the SDN Controller. Multiple path selection algorithms and weighting functions were studied, a small selection is discussed in the following sections.

A. Preprocessing and KPI scaling

Before calculating the weights, pre-processing is performed. First, edges with a KAV lower than the amount of requested keys are removed. Then, it is validated that the source and destination node can still be connected.

To give all KPIs an equivalent contribution they are normed from 1 to 10. This is done because the KAV is usually in the range of thousands of bytes, while edge usage and hop count are much smaller. The corresponding maximum value in the QKDN is assigned a 10, the minimum a 1, intermediate values are scaled as rounded integers. While KAV is always scaled, edge usage and hop count are only scaled if they exceed 10.

B. Dijkstra path finding

Dijkstra's algorithm [29] is a commonly used path selection algorithm and is used in some referenced publications of Section III. Dijkstra inherently penalizes high hop counts, as each additional hop increases the total path cost. Therefore, we do not consider this KPI in the weighting function. However, Dijkstra selects the path with minimal weight, while the KAV is defined so that high values are better. This has to be considered when designing the weighting function.

1) Dijkstra weighting function premise: To guide the formula design, the relation of KAV_N and usage_N of edge_N are compared to the KPIs of edge_M and finally defined as:

Edges with a $\text{KAV}_N \leq 3$ should have a higher weight w than edges with the best KAV, regardless of the usage. An edge_N with the best KAV and the highest usage should be equivalent to an edge_M with 40% of the max KAV and the lowest usage. In other words:

$$\forall\ \text{KAV}_N \leq 3 : f_N(\text{KAV}_N, \text{usage}_N) \geq f_M(10, \text{usage}_M) \quad \text{and } f_N(4,1) = f_M(10,10) \tag{1}$$

2) Linear weighting function: The normed KAV_N is subtracted from 11, so a higher KAV adds less weight. A scaling factor increases the KAV impact compared to the usage. The function is defined as:

$$w_N = 1.5(11 - \text{KAV}_N) + \text{usage}_N \tag{2}$$

This algorithm and weighting function combination was dubbed *linear Dijkstra*.

C. Maximum-Minimum Capacity path finding

The alternative algorithm follows a bottleneck or maximum-minimum capacity approach [30], hereafter dubbed *MM-algorithm*. It selects a path based on the highest minimum weight of an edge in all viable paths between source and destination or in other words the best bottleneck. The algorithm performs the following steps:

1) Obtain all simple paths from source to destination.
2) Give a score to each edge based on Eq. (3).
3) Based on the worst edge of each path select the path with the highest score.
4) If there is a tie, repeat selection for the next worst edge.

In contrast to Dijkstra, this approach does not inherently consider the amount of hops in the path, therefore the hop count KPI is included in Eq. (3):

$$w_N = c_1 \cdot \text{KAV}_N - c_2 \cdot \text{usage}_N - c_3 \cdot \text{hops}_N \tag{3}$$

where c_1, c_2 and c_3 are constants for scaling. The constants of $c_1 = 3$, $c_2 = 2$ and $c_3 = 1$ gave the best results and comply with the reasoning given in Section IV-B.

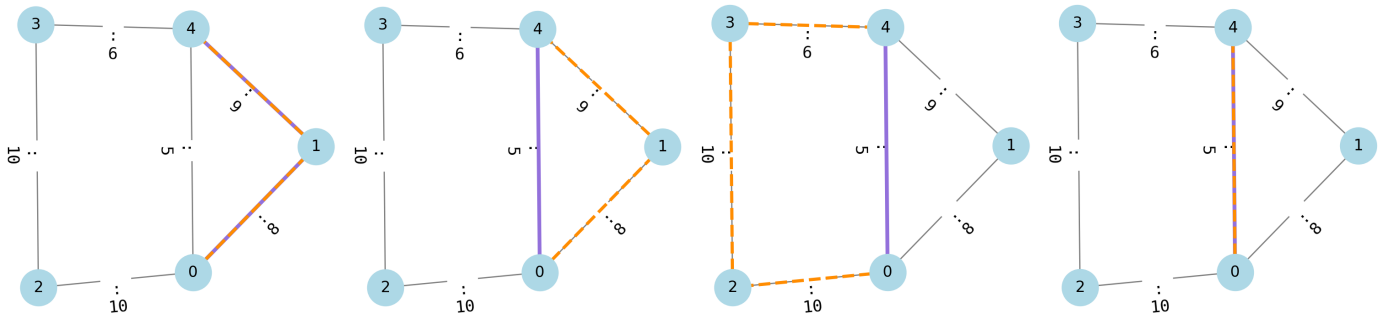
D. Comparison and evaluation

While an SDN for QKDN implementation exists at the authors' institute, as described in [31], the path finding algorithms with different weighting functions were evaluated in a simulation, published on GitHub [11]. This allowed the study of different complex QKDNs and different algorithms. The simulation assumes the pre-processing of Section V-A was already performed. Evaluation based on the authors' experience from projects mentioned in Section IX was conducted.

While complex graphs representing large scale QKDNs have been studied, to present the observed effects in this short paper, an example graph is discussed. It has a source (0) and destination (4) and three different paths:

direct path : A direct path, but it has the lowest KAV.
1 hop : A path with one intermediate node, but good KAV.
2 hops : A path with two intermediate nodes, two edges have the best KAV and one a low KAV.

Considering the priorities of Section IV-B, the direct path with the worst KAV should be avoided in this graph. The path with only one intermediate node and good KAVs seems the best one. Also, the path with two intermediate nodes is viable due to the best KAV, as hops should matter the least.



(a) $N = 1$ (b) $N = 4$ & $N = 6$ (c) $N = 5$ & $N = 8$ (d) $N = 7$

Figure 2: Comparison of Dijkstra and MM-algorithm from node 4 to 0, executed N times. The orange dashed path uses the MM-algorithm of Section V-C and the bold purple uses the linear Dijkstra of Section V-B2.

1) Simulation results: The simulation results for this example are shown in Fig. 2, where the MM-algorithm is compared to the linear Dijkstra. Multiple iterations are shown to determine the influence of increased usage. In the first 4 iterations the 2 node path is used by both, which is the desired behavior. Then the linear Dijkstra stays at the direct link. The MM-algorithm switches to the 3 node path for $N = 5$ and 2 node path at $N = 6$. For $N = 7$ for the first time the MM-algorithm chooses the direct path. For higher N alternating the 3 node, 2 node and 1 node path is chosen by the MM-algorithm.

Other weighting functions have been investigated, e.g. ones based on the inverse of the KAV as some publications in Section III use it. The detailed analysis exceeds the short paper format, but in short those underperformed as they tend to overemphasize the number of hops.

E. Path selection conclusion

The linear Dijkstra and MM-algorithm behaved very close to the desired behavior outlined in Section V-D. The Dijkstra algorithm has the advantage of being efficient and well-studied. The MM-algorithm was studied as an alternative candidate. It showed the most reasonable results, but for bigger networks it is more computationally expensive.

VI. Additional considerations for SDN-QKDN

This section outlines other topics, related to path selection.

A. Key request batching

To reduce the queries to the SDN Controller for path calculation, a key batching optimization is useful where, instead of establishing one key at a time, a batch of keys is allocated for the application. Additionally, the internal key usage for the ITS protection should be considered and modeled as a second batch. So, while the application asks for N keys, the KMS requests allocation of M keys from the SDN with $M = B_{ext} \cdot N + B_{int}$, where B_{ext} is the application batch size multiplier and B_{int} is the internal batch size. Subsequent application key requests don't require a path selection by the SDN Controller, until the batch is depleted.

B. Multi-Path Selection

Multi-path key distribution can increase the security in QKDNs [32], where multiple disjoint paths are chosen. The SDN Controller has to consider all sets of disjoint paths and determine the optimal ones, based on the KPIs.

ETSI GS QKD 015 does not support this feature and would have to be adapted. The `virtual_link_spec` must be made a list of `prev_hop` and `next_hop` instead of single UUID entries. Additionally, a parameter must define the multi-path algorithm and the number of shares.

C. Group Key Distribution

The group key is defined in ETSI GS QKD 014 for applications with more than two participants, for example group chats. The SDN Controller then has to configure multiple paths before the key is distributed in the KMS layer. Similarly to Section VI-B, for this ETSI GS QKD 015 has to be modified to support multiple `next_hop` and `prev_hop` values.

VII. Oblivious Inter-Domain Routing

One of the motivations for this work is to facilitate multi domain QKDNs in the light of EuroQCI. Therefore, not only a consistent path selection is required, also trustworthiness and collaboration between providers, without disclosing internals, must be ensured. A QKDN is a very sensitive infrastructure, requiring high security, therefore the network's current status and metrics can't be shared.

Therefore, we propose oblivious inter-domain routing for QKD. If domains are connected through some border nodes, we envisage the SDN Controllers negotiating a path obliviously by running secure multi-party protocols [33]. By leveraging secure computing technologies, a path can be found without revealing any information about the respective network topologies. This approach maximally preserves the confidentiality of network information while still enabling the SDN to select the globally optimal path. The linear Dijkstra (Section V-B2) is well suited to this problem because it is a greedy algorithm, enabling the providers to pre-compute metrics locally from the source and destination to all border nodes. However, the concept could also be applied to other routing algorithms.

In our Oblivious Inter-Domain Routing (OIDR) proposal, each SDN controller computes aggregated metrics from each internal node to all available border nodes or between border nodes. These metrics serve as secure input for the multi-party protocol run between the SDN controllers to evaluate the path selection obliviously. The oblivious part involves evaluating the sum of the metrics and finding the maximum combination to identify the best path. If only the optimal solution is revealed, the providers do not learn anything about the other party's network internals, yet the optimal overall path is selected and key availability or link usage does not have to be published. This approach preserves the confidentiality of network information as much as possible while enabling the selection of the globally optimal path.

VIII. Conclusion

This work contributes to SDN for QKDN by identifying gaps in the current specification landscape and proposes technical KPIs and algorithms for path selection. These algorithms were simulated and the results are compared. Additional considerations are discussed, such as key batching, multi-path, group keys and oblivious multi-party computation for inter-domain path selection.

IX. Acknowledgment

This work received funding from the Österreichische Forschungsförderungsgesellschaft mbH (FFG) program "Breitband Austria 2030: GigaApp 2. Ausschreibung", project number (PN): FO999917949 ("Q-Crit Austria"). Also, from the European Union's Horizon Europe research and innovation program PN: 101114043 ("QSNP") and from the Digital Europe Program, PNs: 101091642 ("QCI-CAT"), 101091588 ("QUARTER"), and 101091564 ("eCausis").

References

[1] S. S. Gill, O. Cetinkaya, S. Marrone, D. Claudino, D. Haunschild, L. Schlote, H. Wu, C. Ottaviani, X. Liu, S. P. Machupalli *et al.*, "Quantum computing: Vision and challenges," in *Quantum Computing*. Elsevier, 2025, pp. 19–42.

[2] P. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.

[3] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," *Theoretical computer science*, vol. 560, pp. 7–11, 2014.

[4] W. K. Wootters and W. H. Zurek, "A single quantum cannot be cloned," *Nature*, vol. 299, no. 5886, pp. 802–803, 1982.

[5] D. Dieks, "Communication by epr devices," *Physics Letters A*, vol. 92, no. 6, pp. 271–272, 1982.

[6] C. E. Shannon, "Communication theory of secrecy systems," *The Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

[7] "Quantum key distribution networks - key management," International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T), Recommendation ITU-T Y.3803 v1.0, 12 2020.

[8] E. Haleplidis, K. Pentikousis, S. Denazis, J. H. Salim, D. Meyer, and O. Koufopavlou, "Software-Defined Networking (SDN): Layers and Architecture Terminology," RFC 7426, 01 2015.

[9] S. Haji, S. Zeebaree, R. Saeed, S. Ameen, H. Shukur, N. Omar, Z. Ageed, I. Mahmood, and H. Yasin, "Comparison of software defined networking with traditional networking," *Asian Journal of Computer Science and Information Technology*, vol. 9, pp. 1–18, May 2021.

[10] E. Commission. (2023, 04) The european quantum communication infrastructure (euroqci) initiative. [Online]. Available: digital-strategy.ec.europa.eu/en/policies/european-quantum-communication-infrastructure-euroqci

[11] S. Laschet and G. Lendvay. (2026) Sdn key relay for qkdn simulation, v1.0.0. GitHub repository. [Online]. Available: github.com/ait-crypto/sdn-path-finding-simulation

[12] S. Laschet, P. James, G. Lendvay, L. Torresetti, and A. Colombo. (2026) Quicks specification, v2.0.0. GitHub repository. [Online]. Available: github.com/ait-crypto/kms-sdn-agent-interface-specification

[13] "Control interface for software defined networks," European Telecommunications Standards Institute (ETSI), Industry Specification Groups (ISG), Group Specification ETSI GS QKD 015 v2.1.1, 04 2022.

[14] "Orchestration interface for software defined networks," European Telecommunications Standards Institute (ETSI), Industry Specification Groups (ISG), Group Specification ETSI GS QKD 018 v1.1.1, 04 2022.

[15] J. M. Sáez, A. P. Perales, R. C. Palancar, D. R. Lopez, J. F. Chavarria, V. M. Ayuso, and J. P. B. Mendez, "Current status, gaps, and future directions in quantum key distribution standards: implications for industry," in *2024 international conference on Quantum Communications, Networking, and Computing (QCNC)*. IEEE, 2024, pp. 341–345.

[16] P. James, S. Laschet, S. Ramacher, and L. Torresetti, "Key management systems for large-scale quantum key distribution networks," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES '23. New York, NY, USA: Association for Computing Machinery, 2023.

[17] "Quantum key distribution networks - control and management," International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T), Recommendation ITU-T Y.3804 v2.0, 4 2025.

[18] "Quantum key distribution networks - software-defined networking control," International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T), Recommendation ITU-T Y.3805 v1.0, 12 2021.

[19] "Quantum key distribution (qkd); protocol and data format of rest-based key delivery api," European Telecommunications Standards Institute (ETSI), Industry Specification Groups(ISG), Group Specification ETSI GS QKD 014 1.1.1, 02 2019.

[20] "Quantum key distribution (qkd); application interface," European Telecommunications Standards Institute (ETSI), Industry Specification Groups(ISG), Group Specification ETSI GS QKD 004 v2.1.1, 08 2020.

[21] V. Martin, J. P. Brito, L. Ortíz, R. Méndez, J. Buruaga, R. Vicente, A. Sebastian-Lombrana, D. Rincon, F. Perez, C. Sanchez *et al.*, "Madqci: a heterogeneous and scalable sdn-qkd network deployed in production facilities," *npj Quantum Information*, vol. 10, no. 1, p. 80, 2024.

[22] A. A. Fernández, "Diseño e implementación de módulos de seguridad para la red madqci," 2025.

[23] M. Dianati, R. Alléaume, M. Gagnaire, and X. Shen, "Architecture and protocols of the future european quantum key distribution network," *Security and Communication Networks*, vol. 1, no. 1, pp. 57–74, 2008.

[24] C. Bastos, F. Pinto, R. Bacar, G. Anjos, M. Almeida, A. N. Pinto, R. Chaves, T. Dias, J. Afonso, R. Calé, M. Freitas, L. Maia, L. Magalhães, A. Muñiz, R. Cantó, J. P. Brito, J. Ballesta, R. B. Méndez, S. Laschet, S. Ramacher, P. James, L. Torresetti, P. Piscione, A. K. Abdulwahed, P. Giardina, V. Martin, L. Ortiz, A. Pastor, N. Muga, N. Silva, D. López, M. Vieira, C. Escribano, and L. Mengal, "Discretion: First field demonstration of a quantum enabled sdn in the context of a military exercise," in *2025 International Conference on Military Communication and Information Systems (ICMCIS)*, 2025, pp. 1–8.

[25] D. Giemsa, M. Gunkel, T. Johann, S. Pachnicke, R. Boehm, and F. Reuther, "Adapted routing in qkd networks for improved resource utilization," in *Photonic Networks; 24th ITG-Symposium*. VDE, 2023, pp. 1–4.

[26] L.-Q. Chen, M.-N. Zhao, K.-L. Yu, T.-Y. Tu, Y.-L. Zhao, and Y.-C. Wang, "Ada-qkdn: A new quantum key distribution network routing scheme based on application demand adaptation," *Quantum Information Processing*, vol. 20, no. 9, p. 309, 2021.

[27] "Quantum key distribution networks - requirements for quality of service assurance," International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T), Recommendation ITU-T Y.3806 v1.0, 9 2021.

[28] "Quantum key distribution networks - quality of service parameters," International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T), Recommendation ITU-T Y.3807 v1.0, 2 2022.

[29] E. W. Dijkstra, “A note on two problems in connexion with graphs,” in *Edsger Wybe Dijkstra: his life, work, and legacy*, 2022, pp. 287–290.
[30] K. V. Kaymakov and D. S. Malyshev, “On efficient algorithms for bottleneck path problems with many sources,” *Optimization Letters*, vol. 18, no. 5, pp. 1273–1283, 2024.
[31] G. Lendvay, “Software defined networks for large-scale quantum key distribution networks with machine learning enhancement,” Bachelor’s Thesis, IMC University of Applied Sciences Krems, May 2025.
[32] F. Valbusa, T. Lorünser, G. Spini, and S. Laschet, “Relaxing the single point of failure in quantum key distribution networks: An overview of multi-path approaches,” in *Availability, Reliability and Security*, F. Skopik, V. Naessens, and B. De Sutter, Eds. Cham: Springer Nature Switzerland, 2025, pp. 183–200.
[33] “Technical guidelines for secure multi-party computation,” International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T), Recommendation ITU-T X.1770 v1.0, 10 2021.